

35.7. Un nouveau fichier qui appelle BONHOMME : heros.cpp

Un troisième onglet, `heros.cpp`, écrit `montrerHeros(x, y)` avec le `BONHOMME` d'un autre fichier : chaque fichier a son rôle.

1. Ce qu'il faut comprendre

Ce qui est nouveau ici : un troisième fichier, `heros.cpp`. Il contient une fonction, `montrerHeros(x, y)`, qui se sert de `BONHOMME` — **sans l'écrire** : `BONHOMME` vient de `perso_BONHOMME.cpp`.

Chaque fichier a son rôle : `perso_BONHOMME.cpp` dit **à quoi ressemble** le personnage ; `heros.cpp` dit **ce qu'on en fait** ; `principal.cpp` dit **quand** : il ne fait qu'appeler `montrerHeros(72, 64)`.

Pourquoi `heros.cpp` connaît `BONHOMME` : `principal.cpp` verse les deux fichiers. Une fois collés, tout est **un seul programme**, où `BONHOMME` est connu partout. Pour des fonctions et des dessins, l'ordre des deux `#include` ne compte pas : on peut verser `heros.cpp` avant `personnages.cpp`.

Le `#include <sprite16>` a changé d'onglet : c'est `heros.cpp` qui appelle `sprite16()`, c'est donc lui qui écrit la ligne — comme `outils.cpp` au chapitre « Tes propres `#include` ».

Dans l'atelier : un fichier se crée avec « ↗ Agrandir le code », puis « + fichier ».

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Perso>` → [35.1. Perso — dessiner des personnages](#)
- `#include <sprite16>` → [35.2. La fonction `sprite16\(\)` — un personnage de 16 × 16](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **356 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          47 o ← la console, à l'allumage – la mise en route de la console
montrerHeros()     105 o ← main() – ta fonction
main()             21 o ← démarrage – ta fonction
VBlank             8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage      28 o ← image() – attend l'image suivante
AttendreVBlank     16 o ← démarrage – attend que l'écran soit libre
RangerLutins       11 o ← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles       31 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
Tuiles            64 o ← CopierTuiles – 0 lettre sur 43, et 4 tuiles dessinées
DonneesTransfert   10 o ← InstallerTransfert – la petite routine qui copie les lutins
```

Lettres gravées : aucune

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `EffacerCarte`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `Notes`

Fonctions du langage appelées :

```
sprite()      x 4   104 o sur place, dans montrerHeros()  
image()       x 1    6 o sur place, dans main() + AttendreImage (28 o)  
sprite16()    x 1    0 o sur place, dans montrerHeros()
```

Tes fonctions appelées :

```
montrerHeros() x 1   11 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// principal.cpp ne dit plus que QUAND : montrerHeros() fait le reste.
```

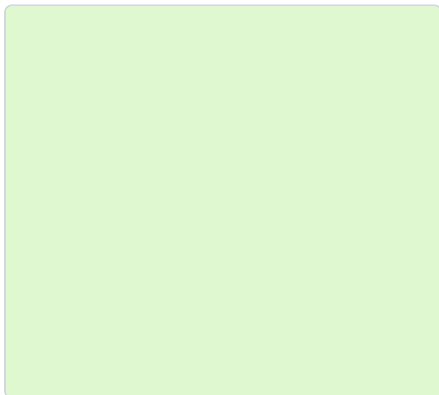
```
#include "personnages.cpp" // la liste : BONHOMME  
#include "heros.cpp"       // montrerHeros() : ce qu'on en fait
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

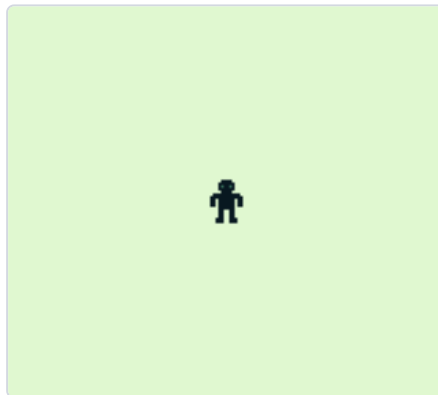
```
int main() {  
    montrerHeros(72, 64); // la fonction de heros.cpp, avec le BONHOMME de perso_BONHOMME.cpp  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le même bonhomme au milieu de l'écran — mais posé par une fonction d'un autre fichier.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ `montrerHeros(72, 64)` pose le lutin 0 en (72, 64) (72, 64)

✓ avec la première tuile de BONHOMME

Ensuite : **35.8. Deux BONHOMME, un seul dessin : le modèle, comme une classe** — chapitre 6 : Dessiner. · gameboy3 — écrire une cartouche Game Boy en C++